

In [1]: *#a music streaming platform wants to develop a personlaized sound recommendation*

```
In [2]: import os

folders = [
    "data",
    "data/raw",
    "data/processed",
    "data/features",
    "models",
    "reports"
]

for folder in folders:
    os.makedirs(folder, exist_ok=True)

print("Folders created successfully")
```

Folders created successfully

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

print("Libraries imported successfully")
```

Libraries imported successfully

```
In [3]: users = pd.read_csv(r"C:\Users\yaswa\Desktop\users.csv")
songs = pd.read_csv(r"C:\Users\yaswa\Desktop\songs.csv")
user_interactions = pd.read_csv(r"C:\Users\yaswa\Desktop\user_intraction.csv")
combined = pd.read_csv(r"C:\Users\yaswa\Desktop\music_recommendation_combined.cs")

print("All datasets loaded successfully")
```

All datasets loaded successfully

```
In [4]: #check size and shap of the data set
print("Users shape:", users.shape)
print("Songs shape:", songs.shape)
print("User Interactions shape:", user_interactions.shape)
print("Combined shape:", combined.shape)
```

Users shape: (1000, 5)  
Songs shape: (500, 16)  
User Interactions shape: (1000, 5)  
Combined shape: (20000, 31)

```
In [5]: #See the first 5 rows
print("USERS")
display(users.head())
```

USERS

|   | user_id | age | gender | preferred_genre | preferred_language |
|---|---------|-----|--------|-----------------|--------------------|
| 0 | U0001   | 54  | Female | Jazz            | English            |
| 1 | U0002   | 44  | Female | Classical       | English            |
| 2 | U0003   | 30  | Male   | Rock            | Spanish            |
| 3 | U0004   | 58  | Female | R&B             | English            |
| 4 | U0005   | 23  | Male   | Electronic      | Spanish            |

```
In [6]: print("SONGS")
display(songs.head())
```

SONGS

|   | song_id | song_name | artist     | genre     | language | duration_sec | popularity | danceabi |
|---|---------|-----------|------------|-----------|----------|--------------|------------|----------|
| 0 | S0001   | Song_0001 | Artist_084 | Classical | Hindi    | 277          | 1          | 0.       |
| 1 | S0002   | Song_0002 | Artist_003 | Classical | Hindi    | 169          | 26         | 0.       |
| 2 | S0003   | Song_0003 | Artist_037 | Hip-Hop   | English  | 304          | 38         | 0.       |
| 3 | S0004   | Song_0004 | Artist_040 | Hip-Hop   | English  | 315          | 38         | 0.       |
| 4 | S0005   | Song_0005 | Artist_072 | Pop       | English  | 221          | 61         | 0.       |

```
In [7]: print("USER INTERACTIONS")
display(user_interactions.head())
```

USER INTERACTIONS

|   | user_id | age | gender | preferred_genre | preferred_language |
|---|---------|-----|--------|-----------------|--------------------|
| 0 | U0001   | 54  | Female | Jazz            | English            |
| 1 | U0002   | 44  | Female | Classical       | English            |
| 2 | U0003   | 30  | Male   | Rock            | Spanish            |
| 3 | U0004   | 58  | Female | R&B             | English            |
| 4 | U0005   | 23  | Male   | Electronic      | Spanish            |

```
In [8]: print("COMBINED")
display(combined.head())
```

COMBINED

|   | interaction_id | user_id | song_id | play_count | liked | skipped | rating | device  | session_d |
|---|----------------|---------|---------|------------|-------|---------|--------|---------|-----------|
| 0 | I000001        | U0158   | S0441   | 3          | 1     | 0       | 5      | Web     |           |
| 1 | I000002        | U0465   | S0029   | 2          | 1     | 0       | 5      | Web     |           |
| 2 | I000003        | U0104   | S0019   | 5          | 1     | 0       | 5      | Desktop |           |
| 3 | I000004        | U0034   | S0479   | 7          | 1     | 0       | 2      | Mobile  |           |
| 4 | I000005        | U0102   | S0073   | 9          | 1     | 0       | 3      | Mobile  |           |

5 rows × 31 columns

```
In [9]: #Check column names
print("Users columns:")
print(users.columns.tolist())

print("\nSongs columns:")
print(songs.columns.tolist())

print("\nUser interactions columns:")
print(user_interactions.columns.tolist())

print("\nCombined columns:")
print(combined.columns.tolist())
```

Users columns:

['user\_id', 'age', 'gender', 'preferred\_genre', 'preferred\_language']

Songs columns:

['song\_id', 'song\_name', 'artist', 'genre', 'language', 'duration\_sec', 'popularity', 'danceability', 'energy', 'loudness', 'speechiness', 'acousticness', 'instrumentalness', 'liveness', 'valence', 'tempo\_bpm']

User interactions columns:

['user\_id', 'age', 'gender', 'preferred\_genre', 'preferred\_language']

Combined columns:

['interaction\_id', 'user\_id', 'song\_id', 'play\_count', 'liked', 'skipped', 'rating', 'device', 'session\_duration\_sec', 'timestamp', 'engagement\_score', 'is\_positive', 'age', 'gender', 'preferred\_genre', 'preferred\_language', 'song\_name', 'artist', 'genre', 'language', 'duration\_sec', 'popularity', 'danceability', 'energy', 'loudness', 'speechiness', 'acousticness', 'instrumentalness', 'liveness', 'valence', 'tempo\_bpm']

```
In [10]: #checking missing values
print("Missing values in USERS:")
print(users.isnull().sum())

print("\nMissing values in SONGS:")
print(songs.isnull().sum())

print("\nMissing values in USER INTERACTIONS:")
print(user_interactions.isnull().sum())
```

Missing values in USERS:

```
user_id      0
age          0
gender       0
preferred_genre  0
preferred_language  0
dtype: int64
```

Missing values in SONGS:

```
song_id      0
song_name    0
artist       0
genre        0
language     0
duration_sec  0
popularity   0
danceability  0
energy       0
loudness     0
speechiness  0
acousticness 0
instrumentalness 0
liveness     0
valence      0
tempo_bpm    0
dtype: int64
```

Missing values in USER INTERACTIONS:

```
user_id      0
age          0
gender       0
preferred_genre  0
preferred_language  0
dtype: int64
```

```
In [11]: #Check duplicate rows
print("Users duplicates:", users.duplicated().sum())
print("Songs duplicates:", songs.duplicated().sum())
print("Interactions duplicates:", user_interactions.duplicated().sum())
```

```
Users duplicates: 0
Songs duplicates: 0
Interactions duplicates: 0
```

```
In [12]: #Check data types
print("USERS")
print(users.dtypes)

print("\nSONGS")
print(songs.dtypes)

print("\nUSER INTERACTIONS")
print(user_interactions.dtypes)
```

```

USERS
user_id      object
age          int64
gender       object
preferred_genre  object
preferred_language  object
dtype: object

```

```

SONGS
song_id      object
song_name    object
artist       object
genre        object
language     object
duration_sec  int64
popularity   int64
danceability  float64
energy       float64
loudness     float64
speechiness  float64
acousticness float64
instrumentalness float64
liveness     float64
valence      float64
tempo_bpm    float64
dtype: object

```

```

USER INTERACTIONS
user_id      object
age          int64
gender       object
preferred_genre  object
preferred_language  object
dtype: object

```

```

In [13]: #Basic dataset information
print("USERS INFO")
users.info()

```

```

USERS INFO
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  -
0   user_id               1000 non-null   object
1   age                   1000 non-null   int64
2   gender                1000 non-null   object
3   preferred_genre       1000 non-null   object
4   preferred_language    1000 non-null   object
dtypes: int64(1), object(4)
memory usage: 39.2+ KB

```

```

In [14]: print("\nSONGS INFO")
songs.info()

```

## SONGS INFO

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 16 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   song_id               500 non-null   object
1   song_name             500 non-null   object
2   artist                500 non-null   object
3   genre                 500 non-null   object
4   language              500 non-null   object
5   duration_sec          500 non-null   int64
6   popularity            500 non-null   int64
7   danceability          500 non-null   float64
8   energy                500 non-null   float64
9   loudness              500 non-null   float64
10  speechiness           500 non-null   float64
11  acousticness          500 non-null   float64
12  instrumentalness       500 non-null   float64
13  liveness              500 non-null   float64
14  valence                500 non-null   float64
15  tempo_bpm             500 non-null   float64
dtypes: float64(9), int64(2), object(5)
memory usage: 62.6+ KB
```

```
In [15]: print("\nUSER INTERACTIONS INFO")
         user_interactions.info()
```

## USER INTERACTIONS INFO

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   user_id               1000 non-null   object
1   age                  1000 non-null   int64
2   gender                1000 non-null   object
3   preferred_genre       1000 non-null   object
4   preferred_language    1000 non-null   object
dtypes: int64(1), object(4)
memory usage: 39.2+ KB
```

```
In [16]: #Statistical summary
         users.describe(include="all")
```

Out[16]:

|               | user_id | age         | gender | preferred_genre | preferred_language |
|---------------|---------|-------------|--------|-----------------|--------------------|
| <b>count</b>  | 1000    | 1000.000000 | 1000   | 1000            | 1000               |
| <b>unique</b> | 1000    | NaN         | 3      | 10              | 5                  |
| <b>top</b>    | U0001   | NaN         | Female | R&B             | English            |
| <b>freq</b>   | 1       | NaN         | 488    | 117             | 454                |
| <b>mean</b>   | NaN     | 38.000000   | NaN    | NaN             | NaN                |
| <b>std</b>    | NaN     | 12.945562   | NaN    | NaN             | NaN                |
| <b>min</b>    | NaN     | 16.000000   | NaN    | NaN             | NaN                |
| <b>25%</b>    | NaN     | 27.000000   | NaN    | NaN             | NaN                |
| <b>50%</b>    | NaN     | 39.000000   | NaN    | NaN             | NaN                |
| <b>75%</b>    | NaN     | 49.000000   | NaN    | NaN             | NaN                |
| <b>max</b>    | NaN     | 60.000000   | NaN    | NaN             | NaN                |

In [17]: `songs.describe(include="all")`

Out[17]:

|               | song_id | song_name | artist     | genre      | language | duration_sec | popularity |
|---------------|---------|-----------|------------|------------|----------|--------------|------------|
| <b>count</b>  | 500     | 500       | 500        | 500        | 500      | 500.000000   | 500.000000 |
| <b>unique</b> | 500     | 500       | 98         | 10         | 5        | NaN          | NaN        |
| <b>top</b>    | S0001   | Song_0001 | Artist_072 | Electronic | English  | NaN          | NaN        |
| <b>freq</b>   | 1       | 1         | 12         | 65         | 229      | NaN          | NaN        |
| <b>mean</b>   | NaN     | NaN       | NaN        | NaN        | NaN      | 252.006000   | 51.866000  |
| <b>std</b>    | NaN     | NaN       | NaN        | NaN        | NaN      | 60.066279    | 27.899908  |
| <b>min</b>    | NaN     | NaN       | NaN        | NaN        | NaN      | 150.000000   | 1.000000   |
| <b>25%</b>    | NaN     | NaN       | NaN        | NaN        | NaN      | 200.000000   | 28.000000  |
| <b>50%</b>    | NaN     | NaN       | NaN        | NaN        | NaN      | 251.000000   | 52.000000  |
| <b>75%</b>    | NaN     | NaN       | NaN        | NaN        | NaN      | 303.000000   | 76.000000  |
| <b>max</b>    | NaN     | NaN       | NaN        | NaN        | NaN      | 360.000000   | 100.000000 |

In [18]: `user_interactions.describe(include="all")`

Out[18]:

|               | user_id | age         | gender | preferred_genre | preferred_language |
|---------------|---------|-------------|--------|-----------------|--------------------|
| <b>count</b>  | 1000    | 1000.000000 | 1000   | 1000            | 1000               |
| <b>unique</b> | 1000    | NaN         | 3      | 10              | 5                  |
| <b>top</b>    | U0001   | NaN         | Female | R&B             | English            |
| <b>freq</b>   | 1       | NaN         | 488    | 117             | 454                |
| <b>mean</b>   | NaN     | 38.000000   | NaN    | NaN             | NaN                |
| <b>std</b>    | NaN     | 12.945562   | NaN    | NaN             | NaN                |
| <b>min</b>    | NaN     | 16.000000   | NaN    | NaN             | NaN                |
| <b>25%</b>    | NaN     | 27.000000   | NaN    | NaN             | NaN                |
| <b>50%</b>    | NaN     | 39.000000   | NaN    | NaN             | NaN                |
| <b>75%</b>    | NaN     | 49.000000   | NaN    | NaN             | NaN                |
| <b>max</b>    | NaN     | 60.000000   | NaN    | NaN             | NaN                |

In [19]: *#Check unique values*

```
for column in users.columns:
    print(column, ":", users[column].nunique())
```

user\_id : 1000

age : 45

gender : 3

preferred\_genre : 10

preferred\_language : 5

In [20]: *for column in songs.columns:*

```
print(column, ":", songs[column].nunique())
```

song\_id : 500

song\_name : 500

artist : 98

genre : 10

language : 5

duration\_sec : 189

popularity : 100

danceability : 364

energy : 380

loudness : 420

speechiness : 231

acousticness : 395

instrumentalness : 382

liveness : 369

valence : 377

tempo\_bpm : 493

In [21]: *for column in user\_interactions.columns:*

```
print(column, ":", user_interactions[column].nunique())
```

user\_id : 1000

age : 45

gender : 3

preferred\_genre : 10

preferred\_language : 5



```
In [22]: print("Users:")
print(users.columns.tolist())

print("\nSongs:")
print(songs.columns.tolist())

print("\nUser Interactions:")
print(user_interactions.columns.tolist())

print("\nCombined:")
print(combined.columns.tolist())
```

Users:

```
['user_id', 'age', 'gender', 'preferred_genre', 'preferred_language']
```

Songs:

```
['song_id', 'song_name', 'artist', 'genre', 'language', 'duration_sec', 'popularity', 'danceability', 'energy', 'loudness', 'speechiness', 'acousticness', 'instrumentalness', 'liveness', 'valence', 'tempo_bpm']
```

User Interactions:

```
['user_id', 'age', 'gender', 'preferred_genre', 'preferred_language']
```

Combined:

```
['interaction_id', 'user_id', 'song_id', 'play_count', 'liked', 'skipped', 'rating', 'device', 'session_duration_sec', 'timestamp', 'engagement_score', 'is_positive', 'age', 'gender', 'preferred_genre', 'preferred_language', 'song_name', 'artist', 'genre', 'language', 'duration_sec', 'popularity', 'danceability', 'energy', 'loudness', 'speechiness', 'acousticness', 'instrumentalness', 'liveness', 'valence', 'tempo_bpm']
```

```
In [23]: display(users.head())
display(songs.head())
display(user_interactions.head())
```

|   | user_id | age | gender | preferred_genre | preferred_language |
|---|---------|-----|--------|-----------------|--------------------|
| 0 | U0001   | 54  | Female | Jazz            | English            |
| 1 | U0002   | 44  | Female | Classical       | English            |
| 2 | U0003   | 30  | Male   | Rock            | Spanish            |
| 3 | U0004   | 58  | Female | R&B             | English            |
| 4 | U0005   | 23  | Male   | Electronic      | Spanish            |

|   | song_id | song_name | artist     | genre     | language | duration_sec | popularity | danceabi |
|---|---------|-----------|------------|-----------|----------|--------------|------------|----------|
| 0 | S0001   | Song_0001 | Artist_084 | Classical | Hindi    | 277          | 1          | 0.       |
| 1 | S0002   | Song_0002 | Artist_003 | Classical | Hindi    | 169          | 26         | 0.       |
| 2 | S0003   | Song_0003 | Artist_037 | Hip-Hop   | English  | 304          | 38         | 0.       |
| 3 | S0004   | Song_0004 | Artist_040 | Hip-Hop   | English  | 315          | 38         | 0.       |
| 4 | S0005   | Song_0005 | Artist_072 | Pop       | English  | 221          | 61         | 0.       |

|   | user_id | age | gender | preferred_genre | preferred_language |
|---|---------|-----|--------|-----------------|--------------------|
| 0 | U0001   | 54  | Female | Jazz            | English            |
| 1 | U0002   | 44  | Female | Classical       | English            |
| 2 | U0003   | 30  | Male   | Rock            | Spanish            |
| 3 | U0004   | 58  | Female | R&B             | English            |
| 4 | U0005   | 23  | Male   | Electronic      | Spanish            |

```
In [24]: interaction_columns = [
    "interaction_id",
    "user_id",
    "song_id",
    "play_count",
    "liked",
    "skipped",
    "rating",
    "device",
    "session_duration_sec",
    "timestamp",
    "engagement_score",
    "is_positive"
]

user_interactions = combined[interaction_columns].copy()

print("Correct interaction dataset created")
display(user_interactions.head())
```

Correct interaction dataset created

|   | interaction_id | user_id | song_id | play_count | liked | skipped | rating | device  | session_d |
|---|----------------|---------|---------|------------|-------|---------|--------|---------|-----------|
| 0 | I000001        | U0158   | S0441   | 3          | 1     | 0       | 5      | Web     |           |
| 1 | I000002        | U0465   | S0029   | 2          | 1     | 0       | 5      | Web     |           |
| 2 | I000003        | U0104   | S0019   | 5          | 1     | 0       | 5      | Desktop |           |
| 3 | I000004        | U0034   | S0479   | 7          | 1     | 0       | 2      | Mobile  |           |
| 4 | I000005        | U0102   | S0073   | 9          | 1     | 0       | 3      | Mobile  |           |

```
In [25]: #Check the new interaction dataset
print("Shape:", user_interactions.shape)
print("\nColumns:")
print(user_interactions.columns.tolist())
```

Shape: (20000, 12)

Columns:

['interaction\_id', 'user\_id', 'song\_id', 'play\_count', 'liked', 'skipped', 'rating', 'device', 'session\_duration\_sec', 'timestamp', 'engagement\_score', 'is\_positive']

```
In [26]: #Check missing values
print("USERS")
print(users.isnull().sum())
```

USERS

|                    |   |
|--------------------|---|
| user_id            | 0 |
| age                | 0 |
| gender             | 0 |
| preferred_genre    | 0 |
| preferred_language | 0 |
| dtype: int64       |   |

```
In [27]: print("\nSONGS")
print(songs.isnull().sum())
```

SONGS

|                  |   |
|------------------|---|
| song_id          | 0 |
| song_name        | 0 |
| artist           | 0 |
| genre            | 0 |
| language         | 0 |
| duration_sec     | 0 |
| popularity       | 0 |
| danceability     | 0 |
| energy           | 0 |
| loudness         | 0 |
| speechiness      | 0 |
| acousticness     | 0 |
| instrumentalness | 0 |
| liveness         | 0 |
| valence          | 0 |
| tempo_bpm        | 0 |
| dtype: int64     |   |

```
In [28]: print("\nINTERACTIONS")
print(user_interactions.isnull().sum())
```

```
INTERACTIONS
interaction_id      0
user_id            0
song_id            0
play_count         0
liked              0
skipped            0
rating             0
device             0
session_duration_sec 0
timestamp          0
engagement_score    0
is_positive        0
dtype: int64
```

```
In [29]: #Check duplicate records
print("Users duplicates:", users.duplicated().sum())
print("Songs duplicates:", songs.duplicated().sum())
print("Interactions duplicates:", user_interactions.duplicated().sum())
```

```
Users duplicates: 0
Songs duplicates: 0
Interactions duplicates: 0
```

```
In [31]: #STEP 18 - Remove duplicate records
users_clean = users.copy()
songs_clean = songs.copy()
interactions_clean = user_interactions.copy()
print("Users duplicates:", users_clean.duplicated().sum())
print("Songs duplicates:", songs_clean.duplicated().sum())
print("Interactions duplicates:", interactions_clean.duplicated().sum())
```

```
Users duplicates: 0
Songs duplicates: 0
Interactions duplicates: 0
```

```
In [32]: #STEP 19 - Check data types
print("USERS")
print(users_clean.dtypes)

print("\nSONGS")
print(songs_clean.dtypes)

print("\nINTERACTIONS")
print(interactions_clean.dtypes)
```

```

USERS
user_id      object
age          int64
gender       object
preferred_genre  object
preferred_language  object
dtype: object

```

```

SONGS
song_id      object
song_name    object
artist       object
genre        object
language     object
duration_sec  int64
popularity   int64
danceability  float64
energy        float64
loudness     float64
speechiness  float64
acousticness float64
instrumentalness float64
liveness     float64
valence      float64
tempo_bpm    float64
dtype: object

```

```

INTERACTIONS
interaction_id  object
user_id        object
song_id        object
play_count     int64
liked          int64
skipped        int64
rating         int64
device         object
session_duration_sec  int64
timestamp      float64
engagement_score float64
is_positive    int64
dtype: object

```

```

In [33]: #STEP 20 - Convert timestamp
interactions_clean["timestamp"] = pd.to_datetime(
    interactions_clean["timestamp"],
    errors="coerce"
)
print(interactions_clean["timestamp"].dtype)

```

datetime64[ns]

```

In [34]: #Check invalid timestamps
print(
    "Invalid timestamps:",
    interactions_clean["timestamp"].isna().sum()
)

```

Invalid timestamps: 0

```

In [35]: #STEP 22 - Check numerical columns
interaction_numeric = [

```

```

    "play_count",
    "liked",
    "skipped",
    "rating",
    "session_duration_sec",
    "engagement_score",
    "is_positive"
]

print(interactions_clean[interaction_numeric].describe())

```

|       | play_count   | liked        | skipped      | rating \     |
|-------|--------------|--------------|--------------|--------------|
| count | 20000.000000 | 20000.000000 | 20000.000000 | 20000.000000 |
| mean  | 3.994800     | 0.64760      | 0.187600     | 3.896800     |
| std   | 1.722417     | 0.47773      | 0.390402     | 1.088535     |
| min   | 1.000000     | 0.00000      | 0.000000     | 1.000000     |
| 25%   | 3.000000     | 0.00000      | 0.000000     | 3.000000     |
| 50%   | 4.000000     | 1.00000      | 0.000000     | 4.000000     |
| 75%   | 5.000000     | 1.00000      | 0.000000     | 5.000000     |
| max   | 15.000000    | 1.00000      | 1.000000     | 5.000000     |

  

|       | session_duration_sec | engagement_score | is_positive  |
|-------|----------------------|------------------|--------------|
| count | 20000.000000         | 20000.000000     | 20000.000000 |
| mean  | 460.085300           | 5.645160         | 0.368400     |
| std   | 253.448271           | 1.685002         | 0.482383     |
| min   | 20.000000            | -0.800000        | 0.000000     |
| 25%   | 241.000000           | 4.500000         | 0.000000     |
| 50%   | 461.000000           | 5.800000         | 0.000000     |
| 75%   | 681.000000           | 6.900000         | 1.000000     |
| max   | 899.000000           | 11.000000        | 1.000000     |

```

In [36]: #STEP 23 - Validate ratings
print(
    interactions_clean["rating"].value_counts().sort_index()
)

```

```

rating
1      776
2     1571
3     3618
4     7011
5     7024
Name: count, dtype: int64

```

```

In [37]: print("Minimum rating:",
              interactions_clean["rating"].min())

print("Maximum rating:",
      interactions_clean["rating"].max())

```

```

Minimum rating: 1
Maximum rating: 5

```

```

In [38]: #STEP 24 - Validate play count
print("Minimum play count:",
      interactions_clean["play_count"].min())

print("Maximum play count:",
      interactions_clean["play_count"].max())

```

Minimum play count: 1  
Maximum play count: 15

```
In [39]: invalid_play_count = interactions_clean[
        interactions_clean["play_count"] < 0
    ]

    print("Invalid play count rows:",
        len(invalid_play_count))
```

Invalid play count rows: 0

```
In [40]: #STEP 25 - Validate song and user IDs
invalid_users = interactions_clean[
    ~interactions_clean["user_id"].isin(users_clean["user_id"])
]

    print("Interactions with invalid users:",
        len(invalid_users))
```

Interactions with invalid users: 0

```
In [42]: #STEP 26 - Clean invalid IDs
interactions_clean = interactions_clean[
    interactions_clean["user_id"].isin(users_clean["user_id"])
]

interactions_clean = interactions_clean[
    interactions_clean["song_id"].isin(songs_clean["song_id"])
]
    print(interactions_clean.shape)
```

(20000, 12)

```
In [43]: #STEP 27 - Check user preferences
    print(
        users_clean["preferred_genre"].value_counts()
    )
```

```
preferred_genre
R&B          117
Hip-Hop      111
Country      106
Electronic   102
Pop          102
Jazz         101
Lo-Fi        96
Classical    89
Indie        89
Rock         87
Name: count, dtype: int64
```

```
In [44]: print(
        users_clean["preferred_language"].value_counts()
    )
```

```
preferred_language
English    454
Hindi      204
Telugu     171
Tamil      92
Spanish    79
Name: count, dtype: int64
```

```
In [45]: #STEP 28 – Check song genres
print(
    songs_clean["genre"].value_counts()
)
```

```
genre
Electronic    65
Jazz           60
Indie          57
Hip-Hop       53
Classical     52
Lo-Fi         48
Rock          44
Country       42
R&B           41
Pop           38
Name: count, dtype: int64
```

```
In [46]: print(
    songs_clean["language"].value_counts()
)
```

```
language
English    229
Hindi      108
Telugu      74
Tamil       49
Spanish     40
Name: count, dtype: int64
```

```
In [48]: #STEP 29 – Save the cleaned datasets
import os

os.makedirs("data/processed", exist_ok=True)
users_clean.to_csv(
    "data/processed/users_clean.csv",
    index=False
)

songs_clean.to_csv(
    "data/processed/songs_clean.csv",
    index=False
)

interactions_clean.to_csv(
    "data/processed/user_interactions_clean.csv",
    index=False
)
```

```
In [49]: print("Cleaned datasets saved successfully!")
```

```
Cleaned datasets saved successfully!
```



```
In [51]: #STEP 30 – Create the master dataset
master_df = interactions_clean.merge(
    users_clean,
    on="user_id",
    how="left"
)
master_df = master_df.merge(
    songs_clean,
    on="song_id",
    how="left"
)
```

```
In [52]: #STEP 31 – Check master dataset
print("Master dataset shape:")
print(master_df.shape)
```

Master dataset shape:  
(20000, 31)

```
In [53]: display(master_df.head())
```

|   | interaction_id | user_id | song_id | play_count | liked | skipped | rating | device  | session_d |
|---|----------------|---------|---------|------------|-------|---------|--------|---------|-----------|
| 0 | I000001        | U0158   | S0441   | 3          | 1     | 0       | 5      | Web     |           |
| 1 | I000002        | U0465   | S0029   | 2          | 1     | 0       | 5      | Web     |           |
| 2 | I000003        | U0104   | S0019   | 5          | 1     | 0       | 5      | Desktop |           |
| 3 | I000004        | U0034   | S0479   | 7          | 1     | 0       | 2      | Mobile  |           |
| 4 | I000005        | U0102   | S0073   | 9          | 1     | 0       | 3      | Mobile  |           |

5 rows × 31 columns



```
In [54]: print(master_df.columns.tolist())

['interaction_id', 'user_id', 'song_id', 'play_count', 'liked', 'skipped', 'rating', 'device', 'session_duration_sec', 'timestamp', 'engagement_score', 'is_positive', 'age', 'gender', 'preferred_genre', 'preferred_language', 'song_name', 'artist', 'genre', 'language', 'duration_sec', 'popularity', 'danceability', 'energy', 'loudness', 'speechiness', 'acousticness', 'instrumentalness', 'liveness', 'valence', 'tempo_bpm']
```

```
In [55]: #STEP 32 – Save master dataset
master_df.to_csv(
    "data/processed/master_music_dataset.csv",
    index=False
)

print("Master dataset saved successfully!")
```

Master dataset saved successfully!